

Giải thích chi tiết: 3 công trình liên quan về bộ nhớ dài hạn cho LLM Agent

Sep 24, 2026 · @Dung Nguyen

Tài liệu này giải thích sâu cả 3 bài báo dùng trong slide khảo sát (APEX-MEM, LightMem, RMM) — đọc trực tiếp từ bản toàn văn, không chỉ tóm tắt. Mục tiêu là để hiểu **cơ chế hoạt động thật sự**, không chỉ nhớ con số.

Bản chất 3 bài, nói cho dễ hiểu

Đọc phần này là nắm được 80% câu chuyện. Các phần sau chỉ là chi tiết và số liệu.

Vấn đề chung cả ba bài đang giải

Mô hình ngôn ngữ quên sạch sau mỗi cuộc trò chuyện. Muốn nó nhớ, phải nhét lại lịch sử cũ vào — nhưng lịch sử qua hàng chục buổi chat thì quá dài, nhét hết vừa không vừa vừa rất tốn tiền.

Nên phải có một **bộ nhớ**: lúc nói chuyện thì ghi lại những gì đáng nhớ, lúc được hỏi thì lấy đúng thứ cần ra. Cả ba bài đều trả lời đúng hai câu hỏi này: **ghi cái gì** và **lấy ra bằng cách nào**. Khác nhau là ở chỗ mỗi bài đặt trọng tâm vào một câu khác nhau.

APEX-MEM — ghi như sổ kế toán

Thay vì lưu nguyên đoạn chat, nó tách ra thành từng mẩu thông tin rõ ràng, kiểu “An làm ở công ty X, từ tháng 3 đến tháng 9”, rồi đổ vào bảng như một file Excel.

Điểm đặc biệt: khi thông tin thay đổi, nó **không xóa dòng cũ**. Nó ghi thêm dòng mới kèm mốc thời gian, giống sổ kế toán không được tẩy xóa. Vậy nên hai thông tin mâu thuẫn nhau vẫn nằm cạnh nhau trong bảng, mỗi cái đúng trong một khoảng thời gian riêng. Lúc bị hỏi, một trợ lý tự viết câu lệnh lọc ra dòng đúng thời điểm.

- **Được**: trả lời chuẩn nhất những câu có yếu tố thời gian, kiểu “lúc đó anh ấy đang làm ở đâu”.
- **Mất**: tốn rất nhiều lượt gọi mô hình, và phải dùng mô hình thật giỏi mới viết đúng câu lệnh đó.

LightMem — ghi kiểu tiết kiệm

Các hệ khác có một tật tốn kém: cứ mỗi câu nói lại gọi mô hình một lần để tóm tắt. LightMem tránh điều đó bằng ba mẹo:

1. **Nén bớt chữ thừa trước đã** — bỏ những từ không mang thông tin.

2. **Gom nhiều câu cùng chủ đề vào một rổ, đầy rổ mới gọi mô hình một lần** — thay vì gọi liên tục.
3. **Việc dọn dẹp để lúc rảnh làm** — lúc đang chat chỉ ghi thêm cho nhanh, chuyện gộp trùng, dọn mâu thuẫn thì xử lý sau.
 - **Được:** rẻ hơn các hệ khác khoảng 30 lần mà điểm vẫn nhỉnh hơn.
 - **Mất:** nó chỉ lưu bản tóm tắt chứ không lưu thông tin có cấu trúc như APEX-MEM, nên câu hỏi cần chính xác mốc thời gian thì nó yếu.

RMM — tự học cách tìm

Hai bài trên lo chuyện **ghi**. RMM lo chuyện **tìm**, vì tìm kiếm thông thường hay lầy nhảm đoạn không liên quan. Nó thêm một bộ lọc nhỏ để xếp lại thứ tự những gì tìm được.

Câu hỏi khó là: làm sao biết bộ lọc xếp đúng hay sai để mà dạy nó? Mẹo của họ: bắt mô hình khi trả lời phải ghi rõ “*tôi đã dùng mẫu nhớ số 3 và số 7*”. Mẫu nào được dùng thì thưởng, không được dùng thì phạt. Vậy là có sẵn đáp án để dạy bộ lọc mà không tốn thêm lượt gọi nào.

Chữ “phản tư” trong bài chỉ có nghĩa là “nhìn lại”. Họ chia làm hai lúc nhìn: *nhìn tới* là cuối mỗi buổi chat thì ngồi tổng kết lại xem buổi này có gì đáng lưu; *nhìn lại* là sau mỗi câu trả lời thì rút kinh nghiệm xem lúc nãy lầy lắt đúng thứ cần chưa. Chỉ vậy thôi, không có gì trừu tượng hơn.

- **Được:** cải thiện được khâu tìm mà gần như không tốn thêm chi phí.
- **Mất:** muốn điểm cao phải dùng mô hình tìm kiếm rất nặng. Và nếu dạy sai chỗ — dạy thẳng vào bộ tìm kiếm thay vì vào bộ lọc — thì kết quả còn tệ hơn lúc chưa dạy gì.

Tóm lại trong một câu mỗi bài

Bài	Một câu
APEX-MEM	Ghi thật kỹ thành bảng có mốc thời gian, không bao giờ xoá — chính xác nhất nhưng đắt nhất
LightMem	Gom lại rồi mới ghi, dọn dẹp để sau — rẻ nhất nhưng ghi thô
RMM	Ghi vừa phải, bù lại bằng việc tự học cách tìm cho đúng hơn

Thuật ngữ và ký hiệu

Đọc phần này trước. Mọi thuật ngữ xuất hiện trong slide và trong tài liệu đều nằm ở đây.

Trước hết, một phân biệt hay bị hỏi: APEX-MEM, LightMem và RMM **không phải mô hình** mà là **hệ thống bộ nhớ** xây quanh một mô hình ngôn ngữ. Mô hình là mạng nơ-ron đã huấn luyện, có trọng số — ở đây là GPT-5, Claude 4.5 Sonnet, GPT-4o, Gemini-1.5-Flash, Qwen3-30B (sinh câu trả lời), GTE-Qwen2-7B và Contriever (nhúng văn bản), LLMingua-2 (nén văn bản). Còn ba bài trên là kiến trúc quy định lưu bộ nhớ ở đâu, dạng gì, khi nào gọi mô hình và truy xuất ra sao; bản thân chúng không được huấn luyện. Đó là lý do đổi Claude sang GPT-4o thì APEX-MEM rơi từ 86,2% xuống 75,0% — kiến trúc giữ nguyên, chỉ thay mô hình bên dưới. Ngoại lệ duy nhất: RMM có một reranker MLP được huấn luyện thật, nhưng đó chỉ là một bộ phận nhỏ bên trong hệ thống.

Thuật ngữ	Nghĩa
LLM	Mô hình ngôn ngữ lớn (large language model)
Agent	Chương trình để LLM tự quyết định gọi công cụ nào, làm gì tiếp, thay vì chỉ sinh chữ một lần rồi thôi
ReAct	Vòng lặp suy luận cộng hành động: nghĩ một bước → gọi công cụ → đọc kết quả → nghĩ tiếp
RAG / NaiveRAG	Cắt văn bản thành đoạn, nhúng thành vector, câu hỏi đến thì lấy vài đoạn giống nhất. NaiveRAG là bản cơ bản nhất, không có bộ nhớ đặc biệt
Benchmark	Bộ dữ liệu chuẩn dùng để đo và so sánh các hệ với nhau
LongMemEval	Benchmark cho bộ nhớ dài hạn: lịch sử hội thoại rất dài qua nhiều phiên, câu hỏi chỉ trả lời đúng nếu nhớ được thông tin rải rác ở các phiên trước. Bản -S là bản rút gọn
LOCOMO	Một benchmark hội thoại dài khác, cùng mục đích
Ablation	Thí nghiệm bỏ đi một thành phần rồi đo điểm tự bao nhiêu, để chứng minh thành phần đó thực sự đóng góp
Baseline	Phương án nền đơn giản dựng ra làm mốc so sánh

Thuật ngữ	Nghĩa
Token	Đơn vị chữ mô hình xử lý, khoảng một từ hoặc một phần từ
Embedder / embedding	Mô hình biến văn bản thành vector để so khớp; embedding là chính vector đó
Retriever	Bộ tìm kiếm theo độ tương đồng vector
Reranker	Bộ sắp xếp lại thứ hạng những gì retriever lấy về
top-K / top-M	Lấy K (hoặc M) mục giống nhất. RMM lấy top-K rồi lọc còn top-M
MLP	Mạng nơ-ron nhỏ vài lớp; đủ nhẹ để huấn luyện trên CPU
Residual	Cộng thẳng điểm của retriever vào đầu ra, để reranker khởi động từ thứ hạng có sẵn thay vì từ ngẫu nhiên
Gumbel-trick	Kỹ thuật lấy mẫu một lựa chọn rời rạc mà vẫn tính được gradient — vì phép chọn rời rạc bình thường làm đứt đường lan truyền gradient
RL / REINFORCE	Học tăng cường. REINFORCE là thuật toán cơ bản nhất: hành động được thưởng thì tăng xác suất chọn lần sau
Recall@5	Tỉ lệ câu hỏi mà trong 5 mục lấy ra có chứa mục chứa đáp án — đo khâu tìm
Accuracy	Tỉ lệ câu trả lời cuối cùng đúng — đo cả hệ . Hai chỉ số này khác nhau, đừng lẫn
Precision / Recall	Trong số thứ hệ chọn ra, bao nhiêu phần đúng (precision); trong số thứ đúng có thật, hệ tóm được bao nhiêu (recall)
Fact	Một mẫu thông tin có cấu trúc: chủ thể – thuộc tính – giá trị
Schema (lược đồ)	Bản thiết kế các bảng và cột trong cơ sở dữ liệu
SQL / SQLite	SQL là ngôn ngữ hỏi dữ liệu; SQLite là hệ cơ sở dữ liệu nhỏ gọn nằm trọn trong một file
Append-only	Chỉ ghi thêm, không sửa không xoá

Thuật ngữ	Nghĩa
t_from, t_to	Khoảng thời gian một fact còn hiệu lực: từ lúc nào đến lúc nào
Θ (Theta) > 0,2	Ngưỡng độ liên quan của một phiên với câu hỏi; dưới ngưỡng thì không dựng đồ thị đầy đủ cho phiên đó
LLMLingua-2	Mô hình nén văn bản: chấm điểm quan trọng từng từ rồi bỏ bớt từ ít quan trọng
Attention	Mức độ mô hình chú ý vào từng từ; chỗ mức chú ý đổi đột ngột thường là chỗ chuyển chủ đề
N, T, r^x , th	Trong công thức LightMem: N là số lượt, T là độ dài trung bình mỗi lượt (token), r^x là phần còn lại sau nén, th là ngưỡng bộ đệm. Số lần gọi LLM = tổng token sau nén chia cho ngưỡng bộ đệm
Soft update / consolidation	Lúc chạy chỉ chèn thêm (soft update); việc hợp nhất, dọn trùng để dành chạy ngoại tuyến (consolidation)
A-MEM, Mem0, Zep, MemoryOS, LangMem	Các hệ bộ nhớ công bố trước đó, dùng làm mốc so sánh trong cả ba bài
ACL / ICLR	Hai hội nghị hàng đầu: ACL về xử lý ngôn ngữ, ICLR về học máy. Bài đăng ở đây là đã qua bình duyệt
arXiv / camera-ready	arXiv là bản tiền ấn bản tác giả tự đăng; camera-ready là bản cuối đăng chính thức tại hội nghị. Số liệu trong tài liệu này lấy từ bản camera-ready
OpenReview	Trang bình duyệt mở của ICLR, nơi đăng bản chính thức và toàn bộ nhận xét phản biện

1. APEX-MEM (ACL 2026, Amazon Science, arXiv 2604.14362)

Bài toán mà nó giải quyết

Các hệ bộ nhớ hội thoại cũ thường làm một trong hai việc tệ: (a) tóm tắt sớm và mất chi tiết quan trọng, hoặc (b) ghi đè thông tin cũ khi có thông tin mới, khiến hệ thống "quên" là thông tin từng đúng ở một thời điểm khác. APEX-MEM giải quyết bằng cách **không bao giờ xoá gì cả** — chỉ thêm, và để việc "ai đúng, đúng lúc nào" cho lúc truy vấn quyết định.

Lược đồ dữ liệu (SQLite, 6+ bảng)

- **entities** — thực thể chuẩn hoá (người, tổ chức, đồ vật...), gắn với 1 trong 35 lớp bản thể học (ontology) kiểu YAGO.
- **properties** — cặp thuộc tính-giá trị của thực thể, có điểm tin cậy riêng.
- **facts** — đơn vị trung tâm: $f = (s, p, v, \delta, [t_from, t_to], c, \varepsilon)$, nghĩa là (chủ thể, thuộc tính, giá trị, kiểu dữ liệu, **khoảng hiệu lực**, **độ tin cậy**, **con trỏ bằng chứng**).
- **events** — sự kiện hội thoại: mốc thời gian, người tham gia, fact liên quan.
- **evidence** — câu gốc trong hội thoại mà fact/event được trích ra (dạng text + lượt + vị trí).
- **turns** — từng lượt hội thoại, có người nói/nghe và mốc thời gian.

Điểm mâu chốt: khi có thông tin mâu thuẫn (VD: "tôi đang sống ở Hà Nội" rồi sau đó "tôi đang sống ở TP.HCM"), hệ thống **không ghi đè** — cả hai fact cùng tồn tại, mỗi cái có khoảng hiệu lực và độ tin cậy riêng. Việc quyết định "cái nào đúng cho câu hỏi này" chuyển hết sang lúc truy vấn (retrieval-time resolution), thay vì lúc xây dựng bộ nhớ (construction-time).

Agent ReAct và 4 công cụ

Một agent kiểu ReAct (Reasoning + Acting) chọn hành động ở mỗi bước, dùng 1 trong 4 công cụ:

1. **SchemaViewer** — xem trước cấu trúc dữ liệu, gợi ý nên dùng công cụ nào tiếp theo.
2. **EntityLookup** — tra thực thể bằng tìm kiếm lai (dense + từ khoá), trả về bản ghi thuộc tính "mới nhất" kèm mốc thời gian.
3. **GraphSQL** — giao diện SQL chỉ-đọc trên các bảng được whitelist, chuyên cho suy luận thời gian: theo dõi fact thay đổi theo thời gian, tính khoảng cách thời gian, truy vết quan hệ nhiều bước.
4. **Search** — tìm kiếm lai ngữ nghĩa + từ khoá, trả về một subgraph gồm thực thể/thuộc tính/sự kiện/lượt hội thoại liên quan.

Agent lập: gọi công cụ → nhận kết quả → quyết định bước tiếp hoặc trả lời cuối. Tối đa 40 bước cho mỗi câu hỏi.

Một chi tiết quan trọng để tiết kiệm chi phí: đồ thị chỉ được dựng đầy đủ trên những **phiên đủ liên quan** tới câu hỏi (ngưỡng độ liên quan $\Theta > 0,2$) — không phải toàn bộ 40-50 phiên đều được xử lý sâu.

Kết quả

Benchmark	Điểm	So sánh
LOCOMO (GPT-5)	88,88% tổng	vs. MIRIX (SOTA trước đó) 85,38%
LongMemEval (Claude Sonnet 4.5 trích xuất + Haiku 4.5 giải quyết thực thể)	86,2%	vs. Nemori 74,6%, session-RAG 72,5%
SealQA-Hard (đa văn bản nhiều)	40,15% (GPT-5)	vs. O3 34,6% — vẫn còn thấp, tác giả tự nhận "còn nhiều chỗ cải thiện"

Ablation quan trọng nhất (lý do nên đọc kỹ bài này)

Cấu hình công cụ	Độ chính xác tổng	Ghi chú
Chỉ SchemaViewer + EntityLookup	77,19%	baseline tối giản
+ GraphQL	79,45% (+2,26)	câu hỏi thời gian: 72,92% → 82,29%; multi-hop cũng tăng
+ Search (đầy đủ)	87,0% (+7,55 so với dòng đầu)	open-domain tăng mạnh nhất: 78%→89,18%

GraphQL-only (bỏ Search) cần **27.282 lượt gọi công cụ** để đạt 79,45%. Hệ đầy đủ dùng **8.260 lượt GraphQL + 8.900 lượt Search + 5.160 lượt EntityLookup (~ 22.300 lượt)** để đạt khoảng 87% — tức phối hợp nhiều công cụ vừa rẻ hơn vừa chính xác hơn là ép một công cụ làm tất cả.

Điểm yếu tự thừa nhận

- **Phụ thuộc mạnh vào LLM giỏi dùng công cụ.** Đổi từ Claude sang GPT-4o: LOCOMO rớt từ 88,88%→86,35%; nghiêm trọng hơn, LongMemEval rớt từ 86,2%→75,0%, vì GPT-4o sinh câu lệnh SQL sai nhiều — tác giả phải thêm ví dụ lỗi vào prompt để vá.
- **Chi phí xây dựng lớn**, đặc biệt ở bước giải quyết thực thể (entity resolution) và trích xuất thuộc tính — không hợp cho triển khai thời gian thực.
- **Độ chính xác của đồ thị phụ thuộc vào chất lượng trích xuất** — lỗi ở bước này lan truyền xuống toàn bộ đồ thị.
- Bản thể học 35 lớp "có thể không phủ hết" các miền chuyên biệt.

- Chỉ xử lý văn bản, chưa có đa phương thức (ảnh, âm thanh).
- **Không có mã nguồn công khai.**

Vì sao không áp nguyên được vào tuần 3

Hạn mức Gemini miễn phí (1.500 lượt/ngày) không kham nổi việc mỗi câu hỏi tốn 10–40 lượt gọi LLM, đặc biệt khi mô hình hiện dùng (Gemini) chưa được kiểm chứng có sinh SQL tốt như Claude hay không.

Phát hiện phụ đáng chú ý

Trong chính bảng kết quả của họ, một **baseline rất đơn giản** (tìm kiếm mở rộng bằng từ khoá lấy từ fact, lấy top-5, mở rộng ra cả phiên chứa nó) đạt **72,5%** trên LongMemEval — cao hơn cả hai hệ thương mại Zep (71,2%) và Mem0 (71,3%), trong khi rẻ hơn nhiều. Đây là mốc so sánh tốt và rẻ để đối chiếu, thay vì so với số vendor tự công bố.

2. LightMem (ICLR 2026, arXiv 2510.18866, mã nguồn: github.com/zjunlp/LightMem)

Bài toán mà nó giải quyết

Các hệ bộ nhớ khác (A-MEM, Mem0...) gọi LLM rất nhiều lần — gần như mỗi lượt hoặc mỗi vài lượt hội thoại một lần — khiến chi phí và độ trễ cao. LightMem đặt câu hỏi: có thể giữ được phần lớn độ chính xác mà giảm mạnh số lần gọi LLM không?

Kiến trúc 3 tầng, lấy cảm hứng từ trí nhớ con người

Light1 — "bộ nhớ cảm giác" (sensory memory):

- Mỗi lượt hội thoại được nén bằng LLMLingua-2 (một mô hình nén văn bản nhỏ, không phải LLM lớn), loại bỏ token có độ "nổi bật" thấp theo một ngưỡng phân vị động ($r = 0,4-0,8$).
- Đồng thời, LightMem dùng chính **ma trận attention** từ các lớp 8–11 của LLMLingua-2 (kết hợp độ tương đồng ngữ nghĩa) để **cắt đoạn hội thoại theo chủ đề** — không cần gọi LLM riêng cho việc này. Độ chính xác cắt đoạn > 80%.
- Bộ đệm giữ tối đa 512 token.

Light2 — bộ nhớ ngắn hạn theo chủ đề:

- Nội dung đã nén được gom vào bộ đệm theo ngưỡng $th = 256-1024$ token.
- **LLM chỉ được gọi khi bộ đệm đầy** — đây là điểm khác biệt cốt lõi so với các hệ gọi theo lượt.

- Mỗi mục nhớ = {chủ đề, embedding, câu hỏi người dùng, câu trả lời mô hình}.

Light3 — bộ nhớ dài hạn:

- **Cập nhật trực tuyến (online):** chỉ chèn thêm mục mới kèm timestamp — không tính toán gì nặng lúc đang hội thoại ("soft update").
- **Hợp nhất ngoại tuyến (offline):** chạy song song sau đó, dùng hàng đợi cập nhật theo độ tương đồng, có ràng buộc thời gian (chỉ mục mới hơn mới được cập nhật mục cũ hơn) để tránh xung đột.

Công thức độ phức tạp

Số lượt gọi API ước lượng: $O(N \cdot r^x \cdot T / th)$, so với baseline gọi mỗi lượt là $O(N)$ — trong đó N là số lượt hội thoại, r là tỉ lệ nén, th là ngưỡng bộ đệm. Đây là lý do hệ này rẻ hơn hẳn về mặt lý thuyết, không chỉ thực nghiệm.

Kết quả (LongMemEval-S, so với A-MEM — baseline mạnh nhất)

Mô hình sinh câu trả lời	LightMem	A-MEM	Chênh lệch
GPT-4o-mini	68,64%	62,60%	+6,04 điểm (câu hình tốt nhất)
Qwen3-30B	72,18%	65,20%	+6,98 điểm (câu hình tốt nhất)

Hiệu quả (kết hợp online+offline): giảm token 10–38 lần, giảm lượt gọi API 3,6–30 lần, tăng tốc runtime 2,9–12,4 lần (so với GPT baseline). Nếu chỉ tính riêng phần online thì còn ấn tượng hơn: giảm token tới 105,9 lần, giảm lượt gọi tới 159,4 lần.

Trên LoCoMo: độ chính xác tăng +6,1 đến +29,29%, hiệu quả token tới 20,92 lần, giảm lượt gọi tới 55,48 lần.

Ablation quan trọng

Bỏ bước cắt đoạn theo chủ đề (topic segmentation) khiến độ chính xác giảm **5,4–6,3 điểm phần trăm** — cho thấy việc cắt đúng ranh giới chủ đề đóng góp thực chất, không chỉ là bước phụ.

Điểm yếu

- **Điểm tuyệt đối không cao.** NaiveRAG (không có gì đặc biệt) cùng cấu hình đã đạt ~61% trên LongMemEval-S — nghĩa là phần "lời" thực chất của toàn bộ kiến trúc phức tạp này chỉ khoảng 7 điểm phần trăm.

- **Đơn vị bộ nhớ là bản tóm tắt theo chủ đề, không phải fact có cấu trúc** — nên các câu hỏi cần độ chính xác cao về slot cụ thể hoặc mốc thời gian chính xác sẽ yếu hơn hệ có cấu trúc như APEX-MEM. Trong một đánh giá độc lập bên ngoài, LightMem chỉ đạt 19,6% ở loại câu hỏi "single-session-assistant" (SS-Asst).
- **LLMLingua-2 (bộ nén cốt lõi) được huấn luyện trên dữ liệu tiếng Anh.** Hành vi khi nén văn bản tiếng Việt (đặc biệt là việc giữ đúng token quan trọng) chưa được tác giả kiểm chứng — đây là suy luận cần tự kiểm tra trước khi áp dụng, không phải kết luận từ bài báo.
- 5 mẫu bị lỗi trong bộ LongMemEval được tác giả loại bỏ khi đánh giá — một thiên lệch nhỏ có lợi cho LightMem trong so sánh.

Nên lấy gì

Ý tưởng "gom nhiều lượt vào bộ đệm, chỉ gọi LLM khi đầy" áp dụng được ngay mà không cần đổi kiến trúc — thay vì gọi LLM sau mỗi lượt hội thoại như thiết kế hiện tại, gom N lượt (hoặc theo ranh giới chủ đề đơn giản) rồi gọi một lần.

3. RMM — "In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents" (ACL 2025, arXiv 2503.08026)

Bài toán mà nó giải quyết

Hầu hết hệ RAG cho hội thoại dài chỉ lo **retrieval tĩnh**: nhúng câu hỏi, tìm top-K, xong. RMM thêm hai cơ chế "phản tư" (reflective): một để quyết định **cách lưu** trí nhớ tốt hơn (nhìn về phía trước — prospective), một để **học cách xếp hạng lại** kết quả tìm kiếm dựa trên tín hiệu thực tế mô hình có dùng trí nhớ đó hay không (nhìn lại — retrospective).

Cơ chế 1: Prospective Reflection (PR) — lúc lưu trí nhớ

- Cuối mỗi **phiên** hội thoại (không phải mỗi lượt), một LLM tách nội dung phiên thành các đơn vị theo **chủ đề**.
- Với mỗi đơn vị mới, so sánh với các mục nhớ đã có bằng độ tương đồng top-K.
- LLM tự quyết định: **hợp nhất (merge)** vào mục đã có, hay **thêm mới (add)**.
- Mỗi mục nhớ cuối cùng = (bản tóm tắt chủ đề, đoạn hội thoại gốc) — bản tóm tắt dùng làm khoá tìm kiếm, đoạn gốc giữ lại để trả lời chính xác.

Cơ chế 2: Retrospective Reflection (RR) — lúc truy xuất

- Một **reranker nhẹ** — một MLP nhỏ có kết nối residual (không phải LLM) — nhận top-K kết quả từ retriever và xếp lại thành top-M cuối cùng.
- Reranker dùng **Gumbel-trick** để lấy mẫu ngẫu nhiên có kiểm soát (giúp huấn luyện RL ổn định hơn).
- **Huấn luyện bằng REINFORCE**: phần thưởng = liệu câu trả lời cuối cùng của LLM có **trích dẫn** mục nhớ đó hay không (+1 nếu có, -1 nếu không). Công thức cập nhật trọng số:
$$\Delta\phi = \eta \cdot (R - b) \cdot \nabla\phi \log P(M_M \mid q, M_K; \phi)$$

với $b = 0,5$ (baseline để giảm phương sai), $\eta = 1e-3$ (tốc độ học).
- **Điểm hay**: câu trả lời và trích dẫn được sinh **trong cùng một lượt gọi LLM** — nghĩa là tín hiệu huấn luyện gần như miễn phí, không tốn thêm lượt gọi riêng để "chấm điểm" retrieval.

Kết quả

Dataset	RMM + GTE-Qwen2-7B	RAG nền
MSC	33,4% METEOR / 57,1% BERTScore	27,5% / 52,1%
LongMemEval	69,8% Recall@5 / 70,4% accuracy	62,4% / 63,6%

Với embedder nhẹ, chạy được trên CPU (Contriever): RMM chỉ đạt **61,2% Recall@5** — kém xa mức 70,4% dùng GTE-Qwen2-7B (một embedder 7 tỷ tham số, cần GPU).

Bảng ablation (quan trọng nhất để hiểu rủi ro của RL)

Cấu hình	METEOR	Recall@5
RAG baseline	24,8%	54,3%
+ PR (Prospective Reflection)	28,6%	57,4%
+ RR không qua reranker — áp RL thẳng lên retriever	20,3%	34,2%
+ RR có reranker (kiến trúc đúng của RMM)	27,5%	58,8%
Full RMM (PR + RR đúng kiến trúc)	30,8%	60,4%

Đây là phát hiện quan trọng nhất của bài báo đối với nhóm: nếu áp dụng RL trực tiếp lên retriever (bỏ qua reranker trung gian), kết quả **sụp xuống dưới cả baseline chưa có RL** ($34,2\% < 54,3\%$) — tác giả cho là do dữ liệu huấn luyện không đủ và hiện tượng "quên

thảm khốc" (catastrophic forgetting) khi RL tác động trực tiếp lên không gian embedding lớn. Reranker nhỏ, tách biệt là thứ giữ cho quá trình học ổn định.

Chất lượng trích dẫn

Precision 87,6%, Recall 85,8%, F1 86,7% — tức là khi mô hình nói "tôi lấy thông tin này từ đây", trích dẫn đó đúng phần lớn thời gian, đủ tin cậy để dùng làm nhật ký vết (audit trail).

Một phân tích phụ thú vị

So sánh độ chi tiết của đơn vị lưu trữ trí nhớ: lưu theo **lượt** chỉ đạt 47% Recall@5, lưu theo **phiên** đạt 69%, mức **tối ưu lý tưởng cho từng câu hỏi** (oracle) đạt 86% — còn **PR một mình** (không cần RR) đã đạt 78%, rất gần oracle. Điều này cho thấy phần lớn giá trị nằm ở việc **chọn đúng đơn vị lưu trữ** (theo chủ đề/phiên), RR chỉ tinh chỉnh thêm.

Một chi tiết hợp với nhóm: tác giả dùng **Gemini-1.5-Flash** làm mô hình sinh câu trả lời, và nó cho kết quả tốt hơn Gemini-1.5-Pro (30,8% vs 24,6% METEOR) — cùng họ mô hình nhóm đang dùng.

Điểm yếu

- Muốn đạt điểm cao cần embedder lớn (GTE-Qwen2-7B) — không hợp với CPU/miễn phí.
- Chi phí huấn luyện RL tăng theo quy mô dữ liệu.
- Chỉ xử lý văn bản, chưa có đa phương thức.
- Có vấn đề riêng tư cần cân nhắc khi lưu trích dẫn nguyên văn.

Nên lấy gì

Hai ý tưởng tách rời nhau, có thể lấy độc lập mà không cần cả hệ thống RL:

1. **Trích xuất theo phiên** (không theo lượt) — giảm mạnh số lần gọi LLM, giống tinh thần LightMem.
2. **Trả lời kèm trích dẫn trong cùng một lượt gọi** — không tốn thêm chi phí, cho ngay dữ liệu để dựng nhật ký vết (theo dõi hệ thống dùng thông tin nào để trả lời).

(RL cho reranker thì **không nên** làm trong tuần 3 — rủi ro cao như bảng ablation cho thấy, cần nhiều dữ liệu huấn luyện và dễ phản tác dụng nếu làm sai cách.)

Bảng so sánh tổng hợp

	APEX-MEM	LightMem	RMM
Đơn vị bộ nhớ	Fact có cấu trúc + khoảng hiệu lực	Tóm tắt theo chủ đề	(Tóm tắt chủ đề, đoạn gốc)
Khi nào gọi LLM để ghi nhớ	Trích xuất lúc xây graph (không phải per-turn)	Khi bộ đệm đầy (~18-30 lần/mẫu)	Cuối mỗi phiên (~1 lần/phiên)
Khi nào gọi LLM để truy xuất	Agent ReAct, 10-40 bước/câu hỏi	1 lần sinh câu trả lời	1 lần (kèm sinh trích dẫn)
Cơ chế đặc trưng	Không xoá, giải quyết xung đột lúc truy vấn	Nén + cắt chủ đề không cần LLM riêng	Reranker RL học từ tín hiệu trích dẫn
Yêu cầu phần cứng	LLM giỏi dùng SQL/tool-use	Nhẹ, CPU được	Cần embedder lớn (GPU) để đạt điểm cao nhất
Có mã nguồn công khai	Không	Có	Không nêu rõ trong bài
Rủi ro lớn nhất khi áp dụng	Phụ thuộc mô hình mạnh, tồn hạn mức	Điểm tuyệt đối chỉ nhỉnh hơn NaiveRAG ~7 điểm	RL sai cách khiến điểm sụp dưới baseline

Khuyến nghị áp dụng cho tuần 4

- Đổi trích xuất theo lượt → theo phiên (RMM/LightMem)** — giảm số lời gọi LLM từ hàng chục xuống 1-vài lần/phiên.
- Thêm điều kiện thời gian vào truy vấn SQL hiện có (APEX-MEM, ablation 72,92%→82,29%)** — không cần đổi sang property graph.
- Trả lời kèm trích dẫn nguồn (RMM)** — không tốn thêm lời gọi LLM.
- Không dùng RL cho reranker ở giai đoạn này** — rủi ro cao (RMM ablation: sụp xuống 34,2% nếu làm sai cách), cần nhiều dữ liệu huấn luyện hơn những gì đang có.
- Không lấy số vendor tự công bố làm mốc so sánh** — dùng baseline đơn giản đã kiểm chứng trong chính bài APEX-MEM (72,5%) hoặc tự đo trên harness của nhóm.

Những con số phải thuộc

Nếu chỉ nhớ được một bảng trong tài liệu này thì nhớ bảng dưới. Mỗi con số đều lấy từ bài báo gốc, và đều là con số có thể bị hỏi ngược.

Con số	Bài	Ý nghĩa khi bị hỏi
88,88% / 86,2%	APEX-MEM	LOCOMO / LongMemEval — điểm cao nhất trong 3 bài
72,92% → 82,29%	APEX-MEM	Ablation mạnh nhất: thêm truy vấn thời gian bằng GraphSQL, đo trên nhóm câu hỏi thời gian
86,2% → 75,0%	APEX-MEM	Đổi Claude sang GPT-4o — điểm rơi 11,2 vì GPT-4o sinh SQL sai
≈22.300 vs 27.282 lượt gọi công cụ	APEX-MEM	Agent phối hợp 3 công cụ tốn ít lượt gọi hơn so với ép chạy GraphSQL mọi lúc (79,45%), mà đạt khoảng 87%
72,5%	APEX-MEM	Baseline đơn giản trong chính bài (mở rộng khoá bằng fact, top-5) — vẫn cao hơn Zep và Mem0
68,64% vs 62,60%	LightMem	LightMem vs A-MEM trên LongMemEval-S, cùng nền GPT-4o-mini
18–30 vs ~987 lượt gọi	LightMem	Chi phí ghi bộ nhớ mỗi mẫu, so với A-MEM (Mem0 ~812)
61%	LightMem	NaiveRAG cùng cấu hình — nên phần lờn thật của LightMem chỉ khoảng 7 điểm
5,4–6,3 điểm	LightMem	Mức giảm khi bỏ bước cắt chủ đề — bằng chứng bước đó có tác dụng thật
69,8% / 70,4%	RMM	Recall@5 / accuracy trên LongMemEval với embedder GTE-Qwen2-7B
61,2%	RMM	Khi đổi sang embedder nhẹ Contriever, trong khi RAG nền đã 58,8%
59,6% → 31,0% (Recall@5: 57,4% → 34,2%)	RMM	Bỏ reranker, áp RL thẳng lên retriever — sụp xuống dưới cả baseline chưa có RL
87,6% / 85,8%	RMM	Precision / recall của phân trích dẫn — cho thấy tín hiệu thưởng đủ sạch để huấn luyện

Con số	Bài	Ý nghĩa khi bị hỏi
93,4% vs 73,8%	Mem0	Số tự công bố vs tái lập độc lập — lý do loại Mem0 khỏi phạm vi khảo sát

Ngân hàng câu hỏi có thể bị hỏi

Mỗi câu trả lời dưới đây đều dựa trên số liệu có trong bài báo gốc. Chỗ nào bài không trả lời được thì nói thẳng là bài không có số — an toàn hơn nhiều so với đoán.

Về phạm vi khảo sát

Sao chỉ khảo sát ba bài? Ba bài này đại diện cho ba hướng thiết kế khác nhau rõ rệt — cấu trúc hoá tối đa, tối ưu chi phí, và học cách truy xuất — nên đủ phủ không gian thiết kế mà không trùng lặp. Thêm bài thứ tư cùng hướng sẽ không thêm thông tin mới.

Sao không có Mem0, Zep hay A-MEM? Ba hệ đó vẫn xuất hiện trong khảo sát, nhưng ở vai trò đối chứng chứ không phải đối tượng chính. Lý do: số Mem0 tự công bố là 93,4% nhưng tái lập độc lập chỉ được 73,8%, nên không dùng làm căn cứ được. A-MEM thì đã bị chính LightMem đo lại và vượt qua.

Ba bài có so sánh trực tiếp được với nhau không? Không, và đây là điểm cần nói rõ. Mỗi bài dùng LLM nền khác nhau (APEX-MEM dùng Claude, LightMem dùng GPT-4o-mini, RMM dùng Gemini-1.5-Flash), embedder khác nhau và cấu hình benchmark khác nhau. Nên không thể đặt 86,2% — 68,64% — 70,4% cạnh nhau rồi kết luận bài nào mạnh hơn. Chỉ so sánh được về thiết kế và về xu hướng chi phí.

Về APEX-MEM

86,2% có đáng tin không? Đáng, nhưng kèm điều kiện. Con số đó gắn chặt với năng lực dùng công cụ của LLM nền: đổi sang GPT-4o thì rơi xuống 75,0% vì sinh SQL sai. Nó là thuộc tính của cặp kiến trúc cộng LLM, không phải của riêng kiến trúc.

Đã có SQL rồi thì cần agent ReAct làm gì? Ablation trả lời trực tiếp: ép chạy GraphQL mọi lúc tốn 27.282 lượt gọi công cụ mà chỉ đạt 79,45%, trong khi để agent tự phối hợp ba công cụ thì tốn khoảng 22.300 lượt mà đạt khoảng 87%. Giá trị của agent nằm ở chỗ nó biết khi nào không cần dùng SQL.

Chỉ ghi thêm không xoá thì số cái phình ra thì sao? Bài không báo cáo chi phí lưu trữ hay tốc độ truy vấn khi số cái lớn dần theo thời gian. Đây là câu hỏi bài chưa trả lời — nên nói thẳng là chưa có số liệu chứ đừng đoán.

Không có mã nguồn thì làm sao kiểm chứng? Đúng, đây là hạn chế lớn nhất về khả năng tái lập. Số liệu chỉ tin được ở mức đã qua bình duyệt của ACL. Muốn tái lập thì phải tự dựng lại từ mô tả lược đồ và bộ công cụ trong bài.

Về LightMem

Nếu NaiveRAG đã 61% thì LightMem hơn ở đâu? Về độ chính xác thì phần lời không lớn, khoảng 7 điểm. Giá trị chính nằm ở chi phí: giảm từ khoảng 987 xuống 18–30 lượt gọi mỗi mẫu, tức khoảng 30 lần. Nếu mục tiêu là triển khai thật thì đó là khác biệt đáng kể; còn nếu chỉ tính điểm benchmark thì đúng là khiêm tốn.

Vì sao cắt chủ đề bằng attention lại hiệu quả? Vì bước nén LLMingua-2 vốn đã phải tính điểm quan trọng cho từng token; điểm đó phản ánh chỗ ngữ cảnh chuyển, nên tận dụng lại được mà không tốn thêm lượt gọi nào. Ablation cho thấy bỏ nó mất 5,4–6,3 điểm.

Hợp nhất ngoại tuyến thì lúc chạy bộ nhớ có bị lỗi thời không? Có, và đó là đánh đổi có chủ ý. Lúc chạy hệ chỉ chèn thêm thật nhanh, nên trong khoảng thời gian trước khi hợp nhất, bộ nhớ có thể chứa mục trùng hoặc mâu thuẫn. Đổi lại là độ trễ khi ghi gần như bằng không.

Về RMM

Thường lấy từ trích dẫn của LLM — LLM trích dẫn sai thì sao? Bài có đo chất lượng trích dẫn: precision 87,6%, recall 85,8%. Tức tin hiệu có nhiều nhưng đủ sạch để huấn luyện. Đây là câu trả lời trực tiếp cho lo ngại đó.

Vì sao phải có reranker riêng, không áp RL thẳng lên retriever? Ablation (Bảng 2 của bài, nền Contriever): áp thẳng thì accuracy tụt từ 59,6% xuống 31,0% và Recall@5 tụt từ 57,4% xuống 34,2%, đều thấp hơn cả RAG nền (58,8% / 54,3%). Lý do: cập nhật RL phá hỏng không gian embedding vốn đã được huấn luyện tốt, trong khi reranker là một lớp mỏng đặt bên trên nên sửa được thứ hạng mà không động vào biểu diễn gốc.

Cần embedder 7B thì có thực tế không? Đây là điểm yếu thật và nên tự nói ra. Với Contriever nhẹ, RMM chỉ còn 61,2% so với RAG nền 58,8%, tức phần lời co lại còn khoảng 2,4 điểm. Mức 70,4% chỉ đạt được khi dùng GTE-Qwen2-7B.

Gumbel-trick dùng để làm gì? Để lấy mẫu rời rạc — tức chọn muốn lấy mục nhớ nào — mà vẫn lan truyền được gradient qua bước chọn đó. Kết hợp với kết nối residual để reranker khởi động từ thứ hạng của retriever thay vì từ ngẫu nhiên.

Câu hỏi mở, khó trả lời nhanh

Nếu phải chọn một hệ để triển khai thật thì chọn cái nào? Tùy ràng buộc. Cần trả lời đúng các câu hỏi có yếu tố thời gian và có sẵn LLM mạnh thì chọn hướng APEX-MEM. Bị giới hạn chi phí gọi API thì chọn hướng LightMem. Có dữ liệu người dùng thật để huấn

luyện và có GPU thì hướng RMM đáng thử, nhưng phải giữ reranker tách riêng.

Cả ba bài có điểm chung nào chưa giải quyết được không? Có ít nhất hai điểm. Thứ nhất, không bài nào đặt vấn đề thông tin sai ngay từ nguồn — cả ba đều giả định những gì người dùng nói là đúng, chỉ khác nhau ở cách xử lý khi hai thông tin đúng mâu thuẫn theo thời gian. Thứ hai, cả ba đều đo trên benchmark hội thoại tiếng Anh, chưa có bằng chứng về đa ngôn ngữ hay miền chuyên biệt.

Nếu bị hỏi một con số mà không nhớ thì sao? Nói là không nhớ chính xác và nêu khoảng, hoặc nói là sẽ kiểm lại trong bài. Đoán một con số sai là cách nhanh nhất để mất độ tin cậy của toàn bộ phần trình bày.

Nguồn đã đối chiếu với bản chính thức tại hội nghị: APEX-MEM — [Proceedings of ACL 2026, Vol. 1: Long Papers, tr. 16470-16489](#); LightMem — [ICLR 2026, poster](#), mã nguồn tại github.com/zjunlp/LightMem; RMM — [Proceedings of ACL 2025, Vol. 1: Long Papers, tr. 8416-8439](#). Mọi con số trong tài liệu này lấy từ bản camera-ready, không phải từ bản arXiv đầu tiên.